SIMATS ENGINEERING

CO1-ASSESSMENT TOOL 1 – APPLICATION-BASED QUESTIONS

Student Name : B. Yogendra

Register Number :192325046

Course Name : Internet Programming

Course Code : CSA4307

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form
action="/register" method="POST">
   <input type="text" name="name">
   <input type="email" name="email">
   <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly..



1.1 Identify appropriate HTML5 semantic elements missing in the structure.

The webpage is missing important HTML5 semantic elements such as:

- <header> – Displays the title of the page.
- <main> – Contains the main content.
- <section> – Groups related content.

- <h1> – Main heading.
- <label> – Provides labels for input fields.
- <footer> – Displays copyright or contact information.

These elements improve the structure, readability, accessibility, and SEO of the webpage.

OUTPUT:



1.2 Modify the structure to make it standards-compliant.

```
<!DOCTYPE html>
<html lang="en">

<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Symposium Registration</title>
</head>

<body>

<header>
<h1>Online Symposium Registration</h1>
</header>

<main>

<section>

<form action="/register" method="POST">

<label for="name">Student Name</label>
<input
type="text"
id="name"
name="name"
placeholder="Enter your name"
```

```
required>

<br><br>

<label for="email">Email</label>
<input
type="email"
id="email"
name="email"
placeholder="Enter your email"
required>

<br><br>

<button type="submit">Submit</button>

</form>

</section>

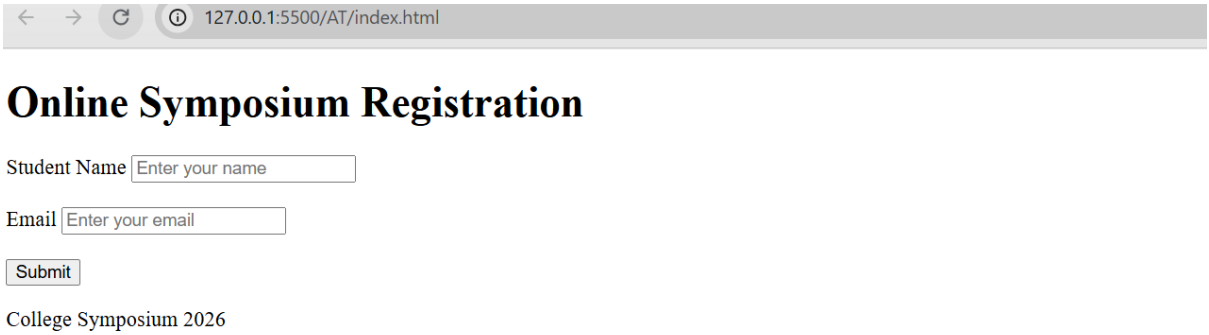
</main>

<footer>
<p>College Symposium 2026</p>
</footer>

</body>

</html>
```

OUTPUT:



1.3 Interpret the HTTP request generated during form submission.

When the student enters their name and email and clicks the Submit button, the browser generates an HTTP POST request and sends it to the server specified in the action attribute of the form.

Form Code:

<form action="/register" method="POST">

This means:

- **POST** → The form data is sent securely in the request body.
- **/register** → The URL where the server receives the registration data.

Example HTTP Request

POST /register HTTP/1.1

Host: www.college.edu

Content-Type: application/x-www-form-urlencoded

name=Rahul&email=rahul@gmail.com

Explanation

- POST indicates that the browser is submitting data to the server.
- /register is the server endpoint that processes the registration.
- Host specifies the website's domain.
- Content-Type tells the server the format of the submitted data.
- The **request body** contains the student's name and email.

1.4 Analyze the HTTP response cycle from server to browser.

After receiving the HTTP POST request, the server processes the submitted data and sends an HTTP response back to the browser.

HTTP Response Cycle

1. The student fills in the registration form.
2. The browser sends an **HTTP POST** request to /register.
3. The server receives the request.
4. The server validates the student's information.
5. The server stores the registration details in the database (if valid).

6. The server generates an HTTP response.
7. The browser receives the response and displays the result to the user.

Example HTTP Response

HTTP/1.1 200 OK

Content-Type: text/html

<h2>Registration Successful!</h2>

Thank you for registering for the Online Symposium.

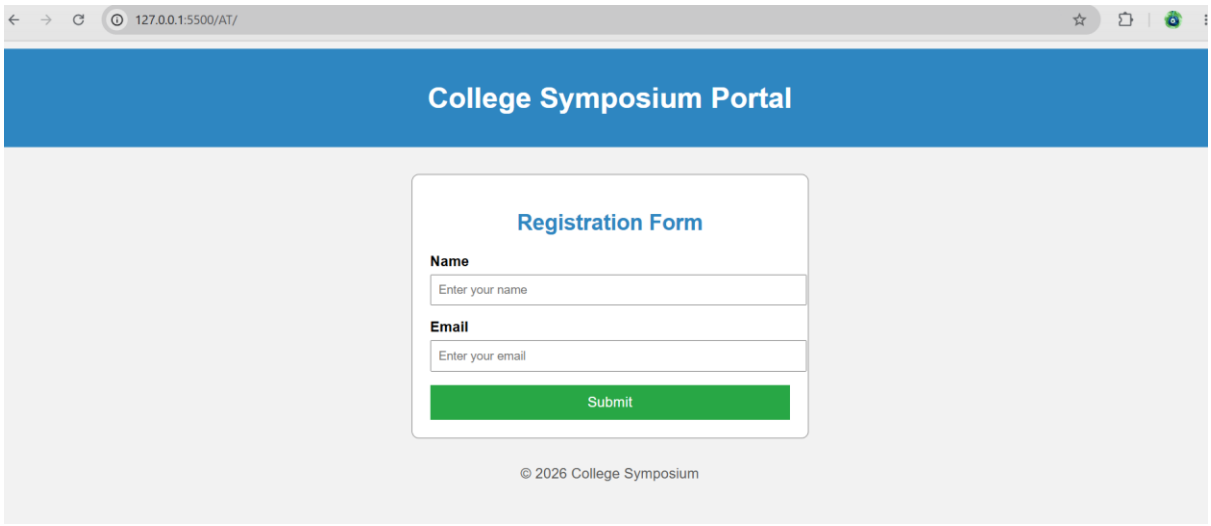
Explanation

- 200 OK indicates that the request was processed successfully.
- The server returns an HTML page containing the confirmation message.
- The browser renders this HTML and displays **"Registration Successful!"** to the user.

1.5 Suggest improvements for usability and accessibility.

The following improvements were made to enhance the usability and accessibility of the Online Symposium Registration Portal:

1. Added semantic HTML5 elements (<header>, <main>, <section>, and <footer>) to provide a clear page structure and improve readability.
2. Added <label> elements for each input field so users can easily understand what information needs to be entered. Labels also improve accessibility for screen readers.
3. Used the required attribute to ensure that the Name and Email fields must be filled before submitting the form.
4. Added placeholder text inside the input fields to guide users on what to enter.
5. Applied basic CSS styling, including colors, spacing, borders, and a centered layout, to make the webpage more attractive and easier to use.
6. Improved the Submit button by giving it a clear label and a hover effect, making it easier to identify and interact with.
7. Added a proper page title and headings so users can immediately understand the purpose of the webpage.


127.0.0.1:5500/AT/
College Symposium Portal
Registration Form
Name
Enter your name
Email
Enter your email
Submit
© 2026 College Symposium

2 Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
<head>
<title>Company</title>
</head>
<body>
<div>
<h1>Welcome
<p>About us
</div>
</body>
</html>
```

2.1:Identify improper nesting and missing closing tags.

Problems

1. <h1> tag is not closed.
2. <p> tag is not closed.
3. The HTML document does not include <!DOCTYPE html>.
4. The <html> tag does not specify the language (lang="en").

2.2: Analyze how different browsers may interpret this structure

Since the HTML contains missing closing tags, different browsers may interpret the page differently.

- Some browsers automatically insert missing closing tags and display the page correctly.
- Other browsers may render the layout differently, causing inconsistent spacing or formatting.
- This can lead to different appearances across browsers such as Chrome, Firefox, Edge, and Safari.

Example:

- **Google Chrome:** Automatically closes missing tags and displays the content normally.
- **Mozilla Firefox:** Also attempts to fix the HTML but may render spacing differently.
- **Microsoft Edge:** Follows HTML5 parsing rules but minor layout differences may occur.

Using properly nested and closed tags ensures consistent rendering in all browsers.

2.3 Identify missing semantic elements

The webpage is missing the following HTML5 semantic elements:

- <header> – For the website heading.
- <main> – Contains the main content.
- <section> – Groups related information.
- <footer> – Displays copyright or contact details.

These elements improve the webpage structure, readability, accessibility, and SEO.

2.4 Provide a corrected W3C-compliant HTML structure

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Company</title>
</head>
<body>
<header>
   <h1>Welcome</h1>
</header>
<main>
   <section>
      <p>About Us</p>
   </section>
</main>
<footer>
   <p>&copy; 2026 Company. All Rights Reserved.</p>
</footer>
</body>
</html>
```

OUTPUT:



2.5: Suggest techniques to ensure cross-browser compatibility

The following techniques help ensure that a webpage works consistently across different browsers:

1. **Use valid HTML5 code** by properly nesting elements and closing all tags.
2. **Use semantic HTML elements** such as <header>, <main>, <section>, and <footer>.
3. **Test the webpage in multiple browsers** (Chrome, Firefox, Edge, Safari) to identify differences.
4. **Use standard CSS properties** that are supported by all modern browsers.
5. **Validate the HTML using the W3C Markup Validator** to detect and fix errors.
6. **Include the viewport meta tag** to ensure proper display on different screen sizes.

3 Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

<form

saction="/submit" method="GET">

<input type="text" name="username">

<input type="password" name="password">

<button type="button">Login</button>

</form>

3.1 Identify issues in the form configuration

Issues

1. The button uses type="button", so clicking it does not submit the form.
2. The form uses the GET method, which is not suitable for login forms because the username and password are sent in the URL.
3. The input fields do not have labels, reducing accessibility.
4. The input fields are not marked as required, allowing empty submissions.
5. There are no placeholders to guide the user.

3.2 Analyze why the HTTP request is not triggered.

The HTTP request is not triggered because the button is defined as:

<button type="button">Login</button>

A button with type="button" performs no action unless JavaScript is attached to it.

To submit the form automatically, the button should be:

<button type="submit">Login</button>

When the user clicks the Login button, the browser submits the form and sends an HTTP request to the URL specified in the action attribute.

3.3 Interpret the role of GET method in this scenario.

The GET method sends form data by appending it to the URL as query parameters.

Example:

GET /submit?username=Rahul&password=abc123 HTTP/1.1

Analysis

- The data is visible in the browser's address bar.
- Sensitive information such as passwords can be exposed.
- URLs may be stored in browser history or server logs.

Therefore, GET is not recommended for login forms. The POST method should be used because it sends the data in the request body instead of the URL.

3.4 Propose a corrected implementation.

```
<!DOCTYPE html>
<html>
<head>
  <title>Login Page</title>

  <style>
    body{
      font-family:Arial;
      background:#f2f2f2;
    }

    .login{
      width:350px;
      margin:50px auto;
      background:white;
      padding:20px;
      border:1px solid gray;
      border-radius:8px;
    }

    h2{
      text-align:center;
      color:#0066cc;
    }

    label{
      font-weight:bold;
    }

    input{
      width:100%;
      padding:8px;
      margin-top:5px;
```

```
            margin-bottom:15px;
        }

        button{
            width:100%;
            padding:10px;
            background:#28a745;
            color:white;
            border:none;
            cursor:pointer;
        }

        button:hover{
            background:#218838;
        }
    </style>

</head>

<body>

<div class="login">

<h2>User Login</h2>

<form action="/submit" method="POST">

<label>Username</label>
<input type="text"
name="username"
placeholder="Enter Username"
required>

<label>Password</label>
<input type="password"
name="password"
placeholder="Enter Password"
required>

<button type="submit">Login</button>

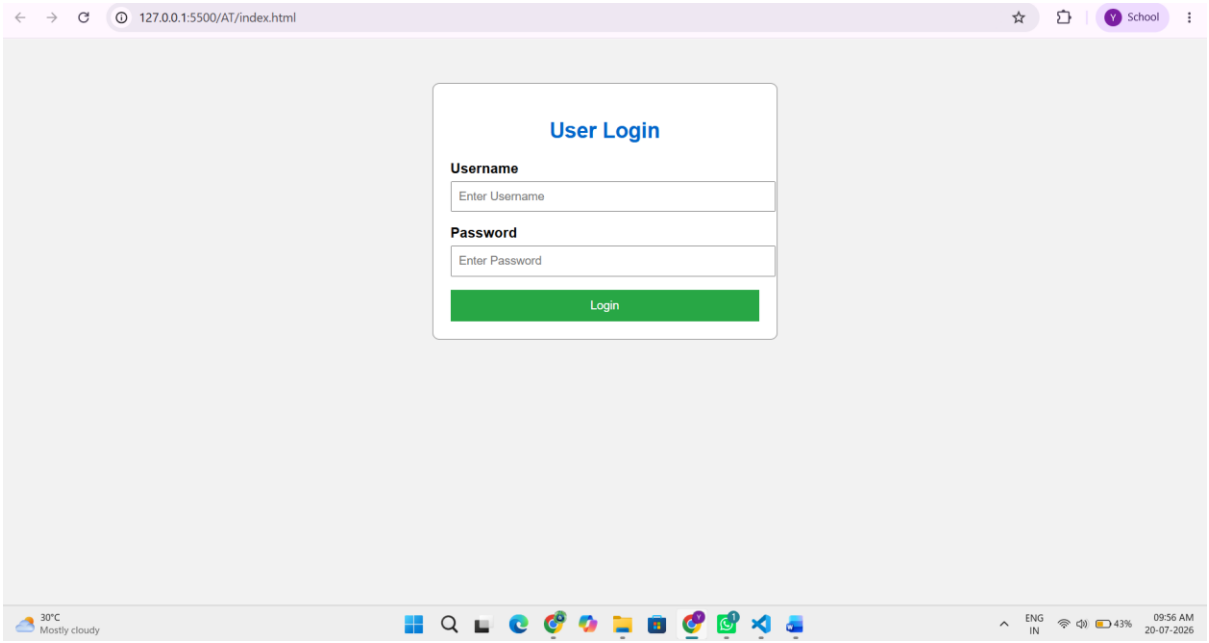
</form>

</div>

</body>
```

</html>

OUTPUT:



3.5 Suggest improvements for secure data transmission.

The following improvements can make the login form more secure:

1. Use the POST method instead of GET so sensitive data is sent in the request body.
2. Use type="submit" for the Login button to ensure the form is submitted correctly.
3. Use HTTPS to encrypt the data during transmission.
4. Add the required attribute to prevent empty submissions.
5. Use labels and placeholders to improve usability and accessibility.
6. Validate user input on both the client side and server side.
7. Never store or display passwords in plain text; passwords should be securely hashed on the server.